

澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU



信息學院
Faculdade de Ciência da Informação e Computação
Faculty of Information Science and Computing

Generative Models

EMEN 7029 & EMEN 7104 - Special Topic In Electromechanical Engineering / Robotics

Qingbiao Li

University of Macau

Review 2
Generative Models 12
Probabilistic Generative Models 21
Bayesian Generative Models 39
Variational Autoencoders 45
Practical Considerations 49
Implementation 54
Hierarchical VAEs 67
Diffusion Models 72

Review

Review



Question: You watch a film. How do you communicate information about the film with a friend?

Answer: Fat panda becomes kung fu master to defend his village from an evil snow leopard.

Question: What is missing?

Answer: Goose father, tortoise teacher, tiger friend, magic scroll, etc

Review

When you discuss films with friends, you summarize them

This is a form of **compression**

$$f\left(\begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_n \end{bmatrix}\right) = \text{Fat panda saves village}$$

Review

Encoders and decoders for images, videos, and music are functions

Neural networks can represent any continuous function

We can use neural networks to represent encoders and decoders

$$f : X \times \Theta \mapsto Z$$

$$f^{-1} : Z \times \Theta \mapsto X$$

We call this an **autoencoder**

Notice no labels Y this time

Review

In supervised learning, humans provide the model with **inputs** $\boldsymbol{X}$ and corresponding **outputs** $\boldsymbol{Y}$

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_{[1]} & \boldsymbol{x}_{[2]} & \dots & \boldsymbol{x}_{[n]} \end{bmatrix}^\top \quad \boldsymbol{Y} = \begin{bmatrix} \boldsymbol{y}_{[1]} & \boldsymbol{y}_{[2]} & \dots & \boldsymbol{y}_{[n]} \end{bmatrix}^\top$$

In unsupervised learning, humans only provide **input**

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_{[1]} & \boldsymbol{x}_{[2]} & \dots & \boldsymbol{x}_{[n]} \end{bmatrix}^\top$$

The training algorithm will learn **unsupervised** (only from $\boldsymbol{X}$)

- Humans do not need to provide labels!

Review

f, f^{-1} may be any neural network

$$\boldsymbol{x} = f^{-1}(f(\boldsymbol{x}, \boldsymbol{\theta}_e), \boldsymbol{\theta}_d)$$

Turn this into a loss function using the square error

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\theta}) = \sum_{j=1}^{d_x} \left(x_j - f^{-1}(f(\boldsymbol{x}, \boldsymbol{\theta}_e), \boldsymbol{\theta}_d)_j \right)^2$$

Forces the networks to compress and reconstruct $\boldsymbol{x}$

Review

Can make many types of autoencoders

- Convolutional autoencoder
- Recurrent autoencoder
- Graph neural network autoencoders
- Transformer autoencoders

The reconstruction objective is useful for compression

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$$

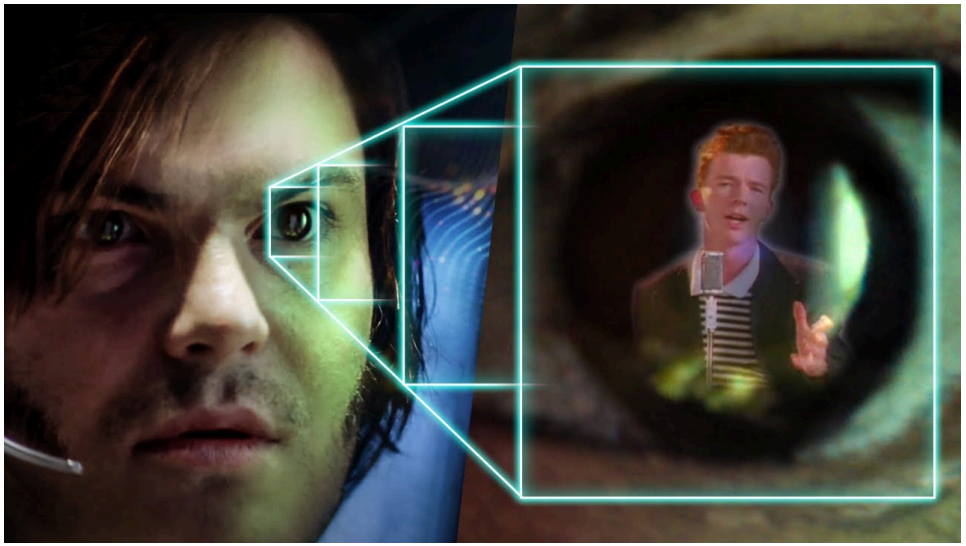
Could we use autoencoders for other tasks?

Review

Task: Fix blurry and noisy image

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\text{blur}\left(\boldsymbol{x}_{[i]} + \boldsymbol{\varepsilon} \right), \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$$





Review

Task: Fix image with missing pixels

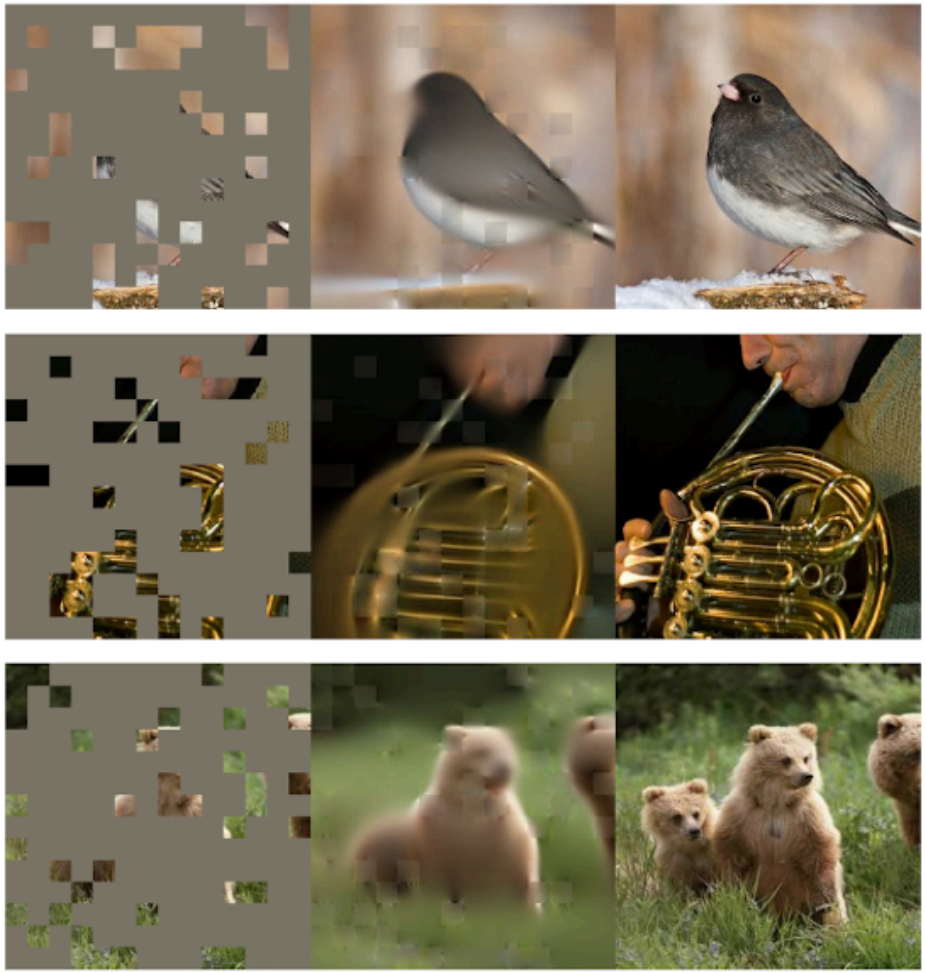
Reconstruction loss $\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$

Sample Bernoulli noise $\boldsymbol{b} \sim \mathcal{B}(0.2)$

Masked reconstruction loss

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]} \odot \boldsymbol{b}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2$$

Review



Understanding emerges from reconstruction objectives

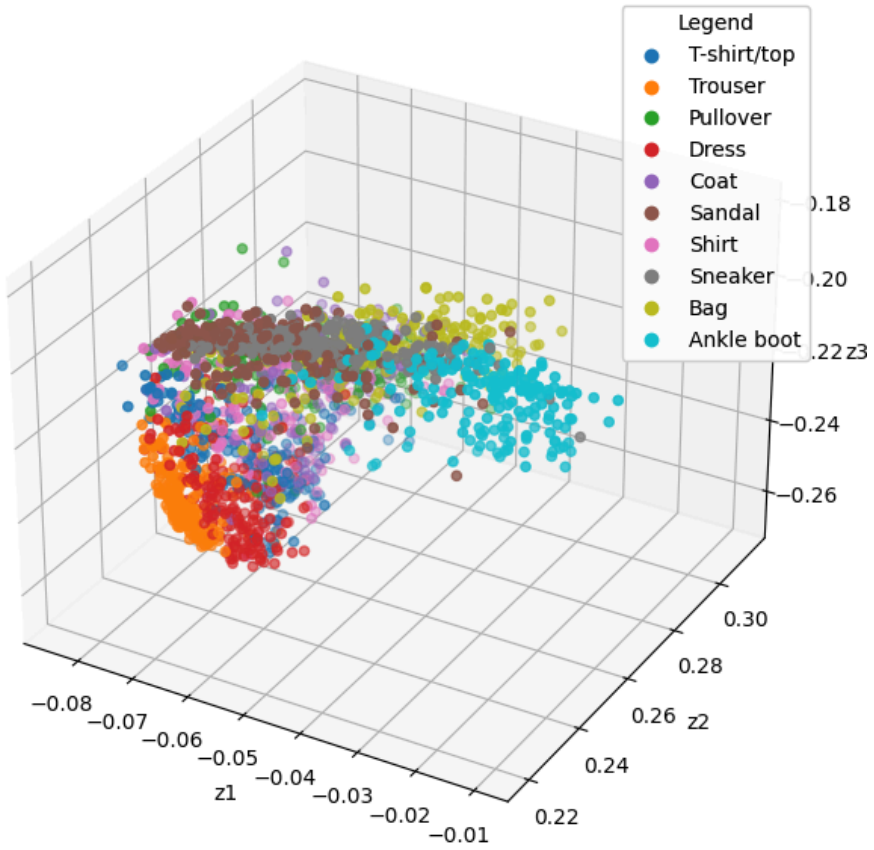
Without labels, autoencoders learn what a bird is, what a bear is

They learn the structure and distribution of the input data X

If the input data X is from our world, then they begin to understand our world

Generative Models

Generative Models



Autoencoder latent space $Z \in \mathbb{R}^3$

Images with similar semantic meaning (sneaker, sandal) cluster in Z

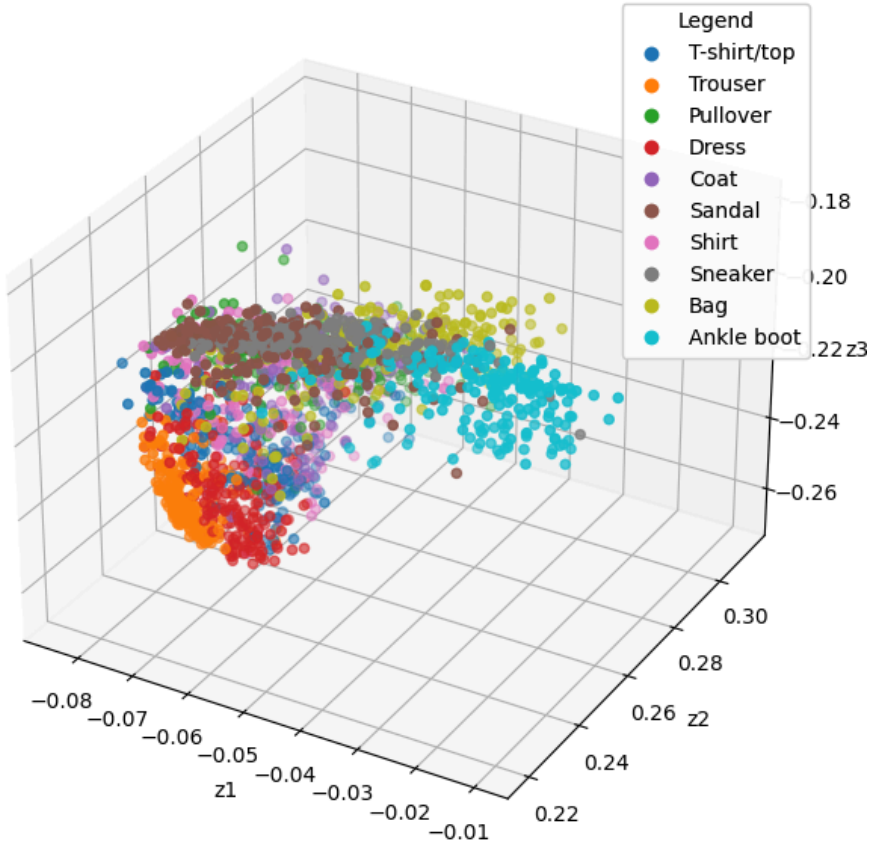
Z forms a semantic manifold

- Each datapoint locally Euclidean

Network learns what sneakers, shirts, etc are

- Can we use this to generate **new** images?

Generative Models



Each $\boldsymbol{z}$ corresponds to $\boldsymbol{x}$ in the dataset

Consider a datapoint $\boldsymbol{x}_{[i]}$ (shirt)

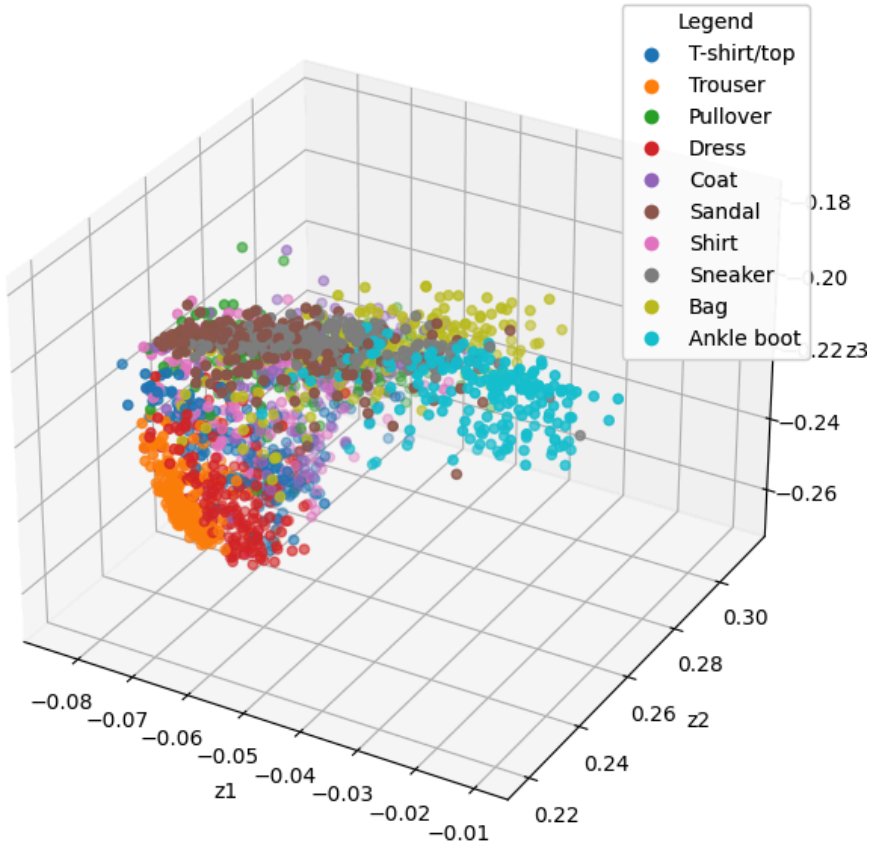
We can encode $\boldsymbol{x}_{[i]}$

$$\boldsymbol{z}_{[i]} = f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e\right)$$

And decode $\boldsymbol{z}_{[i]}$

$$\hat{\boldsymbol{x}}_{[i]} = f^{-1}\left(\boldsymbol{z}_{[i]}, \boldsymbol{\theta}_d\right)$$

Generative Models



$$\boldsymbol{z}_{[i]} = f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e\right)$$

$$\hat{\boldsymbol{x}}_{[i]} = f^{-1}\left(\boldsymbol{z}_{[i]}, \boldsymbol{\theta}_d\right)$$

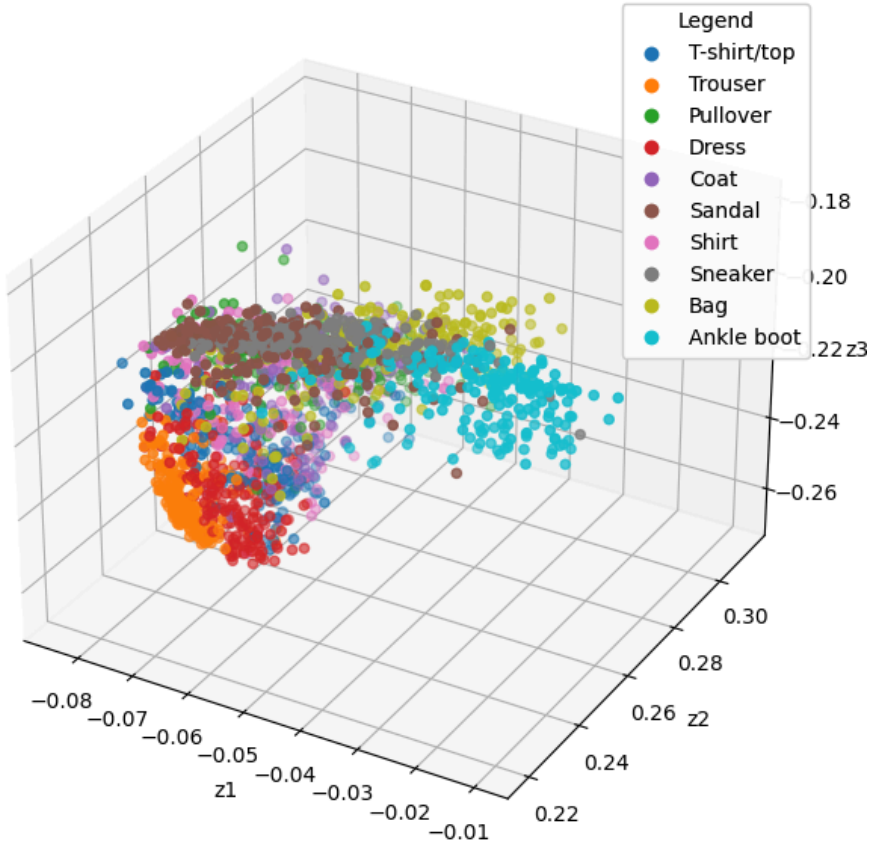
Let us perturb $\boldsymbol{z}$ with noise ε, then decode it

$$f^{-1}\left(\boldsymbol{z}_{[i]} + \varepsilon, \boldsymbol{\theta}_d\right)$$

Question: What happens?

Hint: Z is a semantic manifold (locally Euclidean)

Generative Models



$$f^{-1}\left(\boldsymbol{z}_{[i]} + \varepsilon, \boldsymbol{\theta}_d\right)$$

We will create an image of a new datapoint

- Image not in the dataset
- Semantically similar to $\boldsymbol{x}_i$
  - But **new** and different!

$$\boldsymbol{z}_{[i]} \rightarrow \text{blue shirt}$$

$$\boldsymbol{z}_{[i]} + \varepsilon \rightarrow \text{red shirt}$$

Idea powers generative models

Generative Models

Generating new clothing pictures is not so exciting

But we can apply generative models to many exciting problems

- Train on books, generate new stories
- Train on music, generate new music
- Train on medicine, generate new medicine

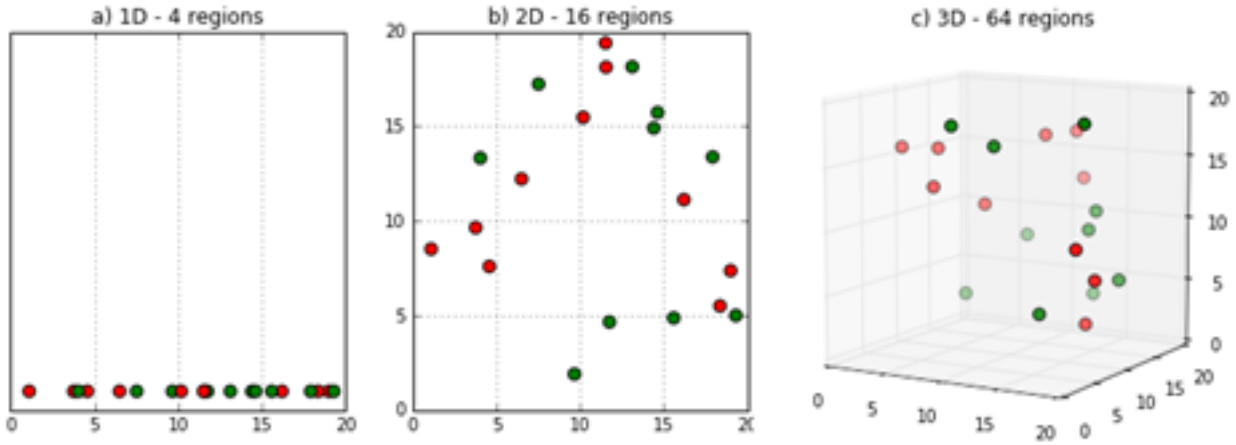
Is it that easy? Train autoencoder on medicine to create new medicines?

Unfortunately, no. We can do it, but not with a simple autoencoder

Generative Models

This autoencoder only works for small d_z

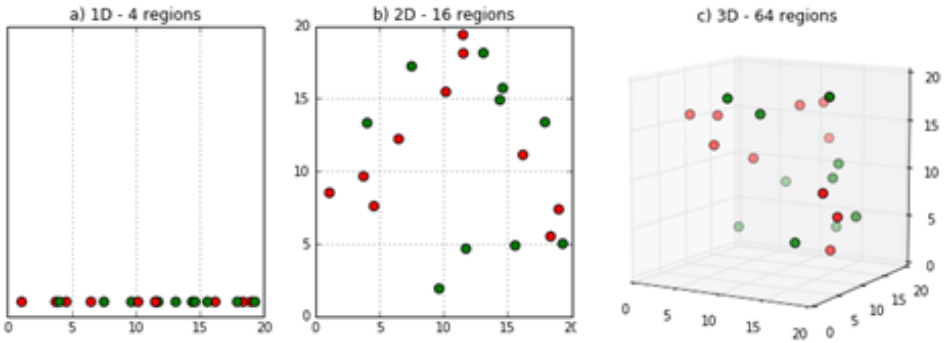
- More interesting problems are high-dimensional (large d_z)



As d_z grows, points spread out (curse of dimensionality)

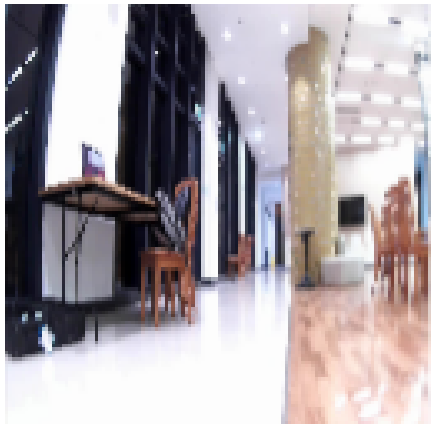
- For large d_z, one cluster for each datapoint

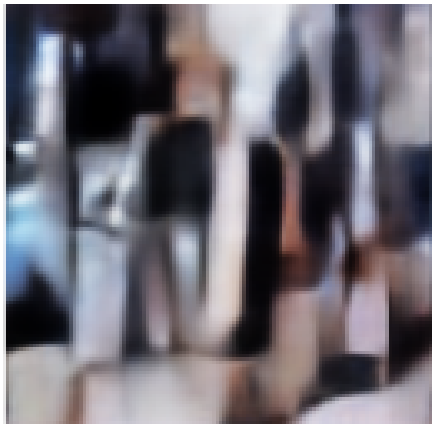
Generative Models



$$f^{-1}(\boldsymbol{z} + \boldsymbol{\varepsilon}); d_z \gg 0$$

Adding noise and decoding does not work for large d_z





Generative Models

Autoencoder latent space has learned structure

- We must enforce some latent structure for generative models
- We use a probabilistic framework for this
- Very active area of research

Probabilistic Generative Models

Probabilistic Generative Models

Question: How do we explain autoencoders learning our world?

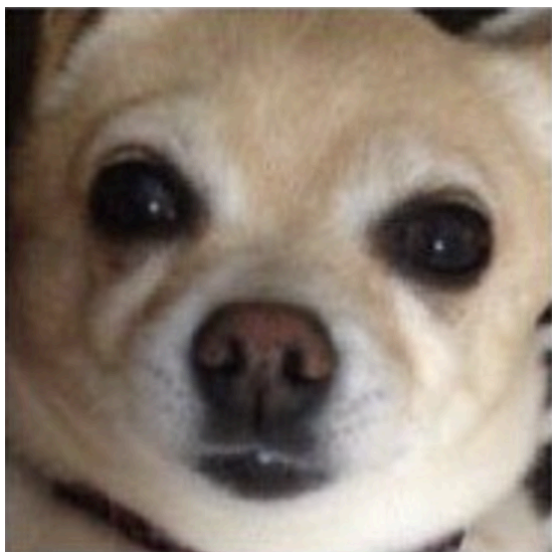
Answer: They implicitly model the **dataset distribution**

- If the dataset contains our world, autoencoders learn the world

Must rethink machine learning to understand generative models

- Most lectures I try to skip/ignore probability theory
- Understanding probability is necessary for generative models
  - Please try your best to pay attention!

Let us consider a dataset of dog pictures



$\boldsymbol{x}_{[i]}$

Domain: $X = [0, 1]^{\{3 \times 28 \times 28\}}$

- How we represent datapoints
- $\boldsymbol{x}_{[i]} \in X$

Dataset (Evidence): $\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_{[1]} & \dots & \boldsymbol{x}_{[n]} \end{bmatrix}^\top$

- Finite collection of n datapoints

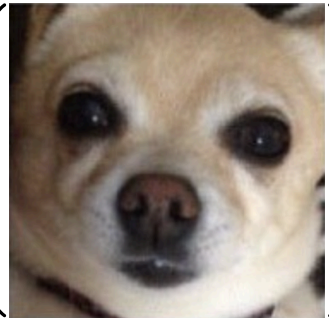
Dataset distribution: $P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right)$

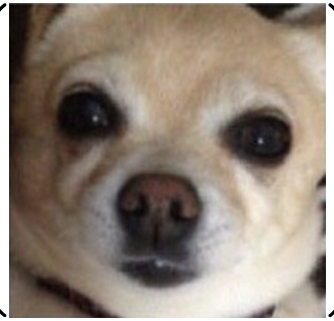
- Probability of uniformly sampling $\boldsymbol{x}_{[i]}$ from $\boldsymbol{X}$
- $\frac{1}{n}$ if $\boldsymbol{x}_{[i]}$ in dataset, else 0

True data distribution: $p_*\left(\boldsymbol{x}_{[i]}\right)$

- How likely is $\boldsymbol{x}_{[i]}$ to exist in the universe?

Probabilistic Generative Models

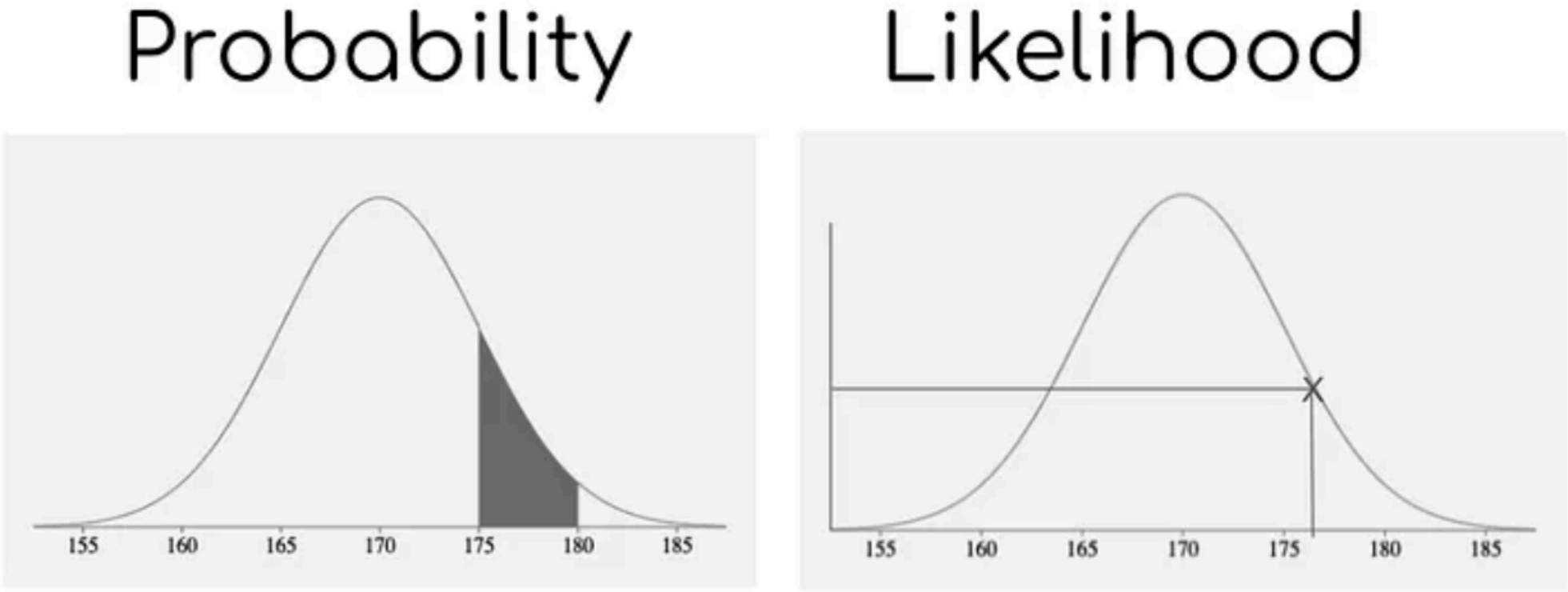
$$P_{\boldsymbol{X}}\left(\text{  }\right) = \frac{1}{n} \qquad P_{\boldsymbol{X}}\left(\text{  }\right) = 0$$

$$p_*\left(\text{  }\right) \begin{matrix} = \\ < \\ > \end{matrix} p_*\left(\text{  }\right)$$

The probability of a point in a continuous distribution is always 0

- For continuous distributions, we consider density not probability

Probabilistic Generative Models



We let small p represent likelihood

Probabilistic Generative Models

We want to approximate a distribution so we can generate samples

Question: $\boldsymbol{x}_{[i]} \sim P_{\boldsymbol{X}}(\boldsymbol{x}; \boldsymbol{\theta})$ or $\boldsymbol{x}_{[i]} \sim p_*(\boldsymbol{x}; \boldsymbol{\theta})$

$P_{\boldsymbol{X}}(\boldsymbol{x}; \boldsymbol{\theta})$ samples finite, existing datapoints, must be $p_*(\boldsymbol{x}; \boldsymbol{\theta})$

Only have $P_{\boldsymbol{X}}(\boldsymbol{x}; \boldsymbol{\theta})$, not $p_*(\boldsymbol{x}; \boldsymbol{\theta})$

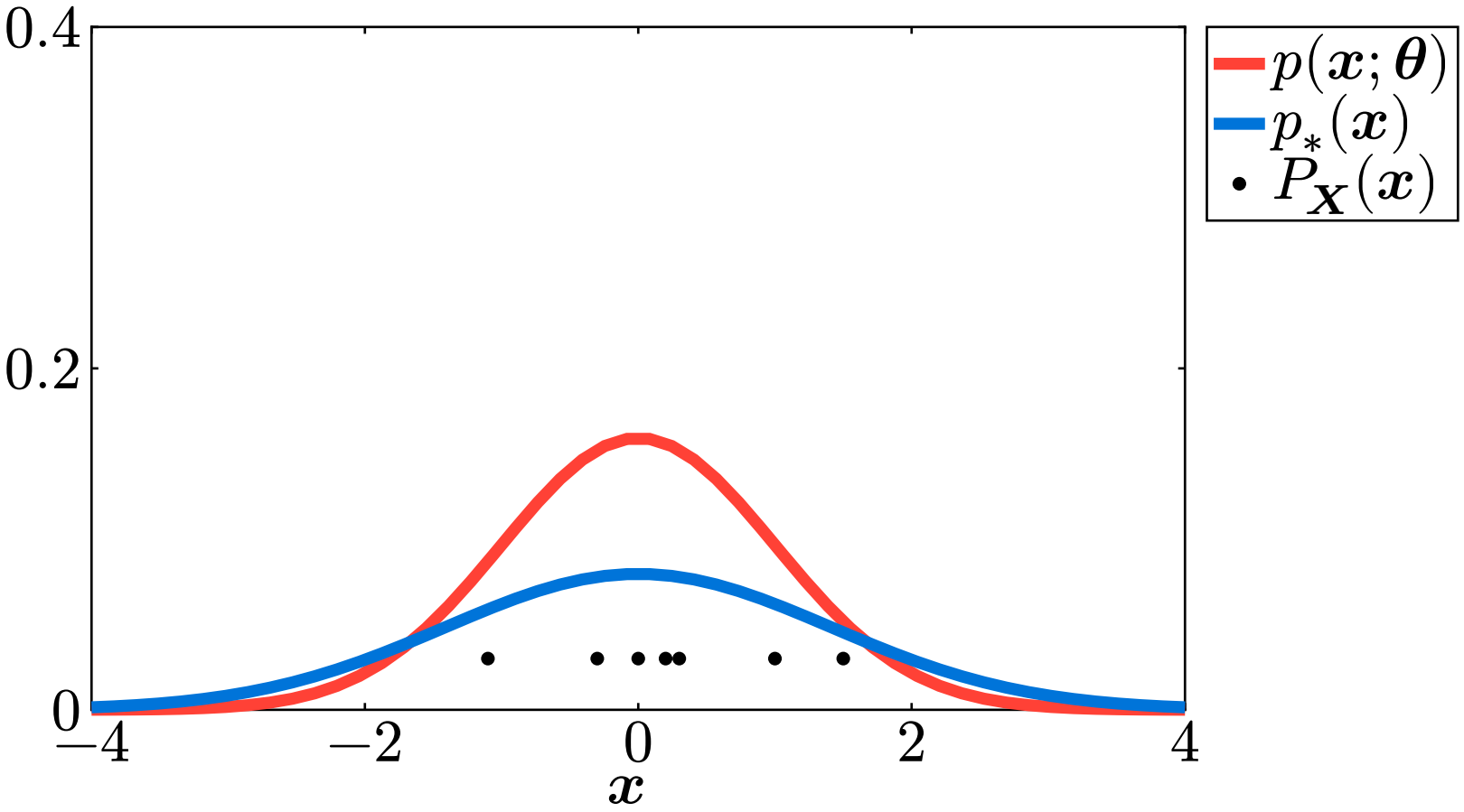
- How to approximate $p_*(\boldsymbol{x}; \boldsymbol{\theta})$?

Key idea: Learn a continuous distribution $p(\boldsymbol{x}; \boldsymbol{\theta})$ using $P_{\boldsymbol{X}}(\boldsymbol{x})$

$$p(\boldsymbol{x}; \boldsymbol{\theta}) = P_{\boldsymbol{X}}(\boldsymbol{x}) \Rightarrow p(\boldsymbol{x}; \boldsymbol{\theta}) \approx p_*(\boldsymbol{x})$$

We call $p(\boldsymbol{x}; \boldsymbol{\theta})$ a **probabilistic generative model**

Probabilistic Generative Models



Probabilistic Generative Models

Truly understanding generative models is very hard

- Took me years, I still do not fully understand

If you understand this concept, all generative models become easy

- Same objective, just different approximation methods
  - GAN
  - VAE
  - Diffusion
  - Normalizing flow
  - Flow matching
  - ...

Probabilistic Generative Models

Let us think about the objective function for a generative model

- We have the dataset distribution $P_{\boldsymbol{X}}(\boldsymbol{x})$
- We have a model that outputs a distribution $p(\boldsymbol{x}; \boldsymbol{\theta})$
- Output distribution must match the dataset distribution

Question: What is our objective? **Hint:** Two distributions

$$\arg\min_{\boldsymbol{\theta}} \ \mathrm{KL}(P_{\boldsymbol{X}}(\boldsymbol{x}), p(\boldsymbol{x}; \boldsymbol{\theta}))$$

Same idea as classification

- Let us see if we can simplify the objective

Probabilistic Generative Models

$$\arg\min_{\boldsymbol{\theta}} \mathrm{KL}(P_{\boldsymbol{X}}(\boldsymbol{x}), p(\boldsymbol{x}; \boldsymbol{\theta}))$$

From the definition of KL

$$\arg\min_{\boldsymbol{\theta}} \mathrm{KL}(P_{\boldsymbol{X}}(\boldsymbol{x}), p(\boldsymbol{x}; \boldsymbol{\theta})) =$$

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \log \frac{P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right)}{p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)}$$

However, we know the probability for all datapoints $P_{\boldsymbol{X}}(\boldsymbol{x})$

Question: What is it? **Answer:** $\frac{1}{n}$ if $\boldsymbol{x}_{[i]}$ in dataset, else 0

Probabilistic Generative Models

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \log \frac{P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right)}{p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)}$$

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \left[\log P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) - \log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)\right]$$

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \log P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) - P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)$$

$$\arg\min_{\boldsymbol{\theta}} \left(\sum_{i=1}^{n} P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \log P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right)\right) - \left(\sum_{i=1}^{n} P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)\right)$$

Probabilistic Generative Models

$$\arg\min_{\boldsymbol{\theta}} \left(\sum_{i=1}^{n} P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \log P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \right) - \left(\sum_{i=1}^{n} P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right) \right)$$

First term is constant and does not depend on $\boldsymbol{\theta}$, can delete it!

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)$$

Very similar to classification loss derivation so far

Probabilistic Generative Models

$$\arg\min_{\boldsymbol{\theta}} -\sum_{i=1}^{n} P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) \log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)$$

Let us think about $P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right)$, can we rewrite it?

- **Hint:** The sum is over the dataset
- **Hint:** What is $P_{\boldsymbol{X}}$ for a datapoint in the dataset?

$$P_{\boldsymbol{X}}\left(\boldsymbol{x}_{[i]}\right) = \frac{1}{n}$$

$$\arg\min_{\boldsymbol{\theta}} -\sum_{i=1}^{n} \frac{1}{n} \log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)$$

Probabilistic Generative Models

$$\arg\min_{\boldsymbol{\theta}} -\sum_{i=1}^{n} \frac{1}{n} \log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)$$

$\frac{1}{n}$ is a constant, does not affect minima

$$\arg\min_{\boldsymbol{\theta}} -\sum_{i=1}^{n} \log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)$$

Probabilistic Generative Models

$$\arg\min_{\boldsymbol{\theta}} -\sum_{i=1}^{n} \log p\Big(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\Big)$$

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log p\Big(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\Big)$$

This is the **negative log likelihood**

- Used all over statistics and machine learning (e.g. LLM)

The log likelihood objective:

- Learns the parameters $\boldsymbol{\theta}$ of a continuous distribution $p(\boldsymbol{x}; \boldsymbol{\theta})$
- That maximizes the likelihood of each datapoint $\boldsymbol{x}_{[i]}$ under the model

Probabilistic Generative Models

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)$$

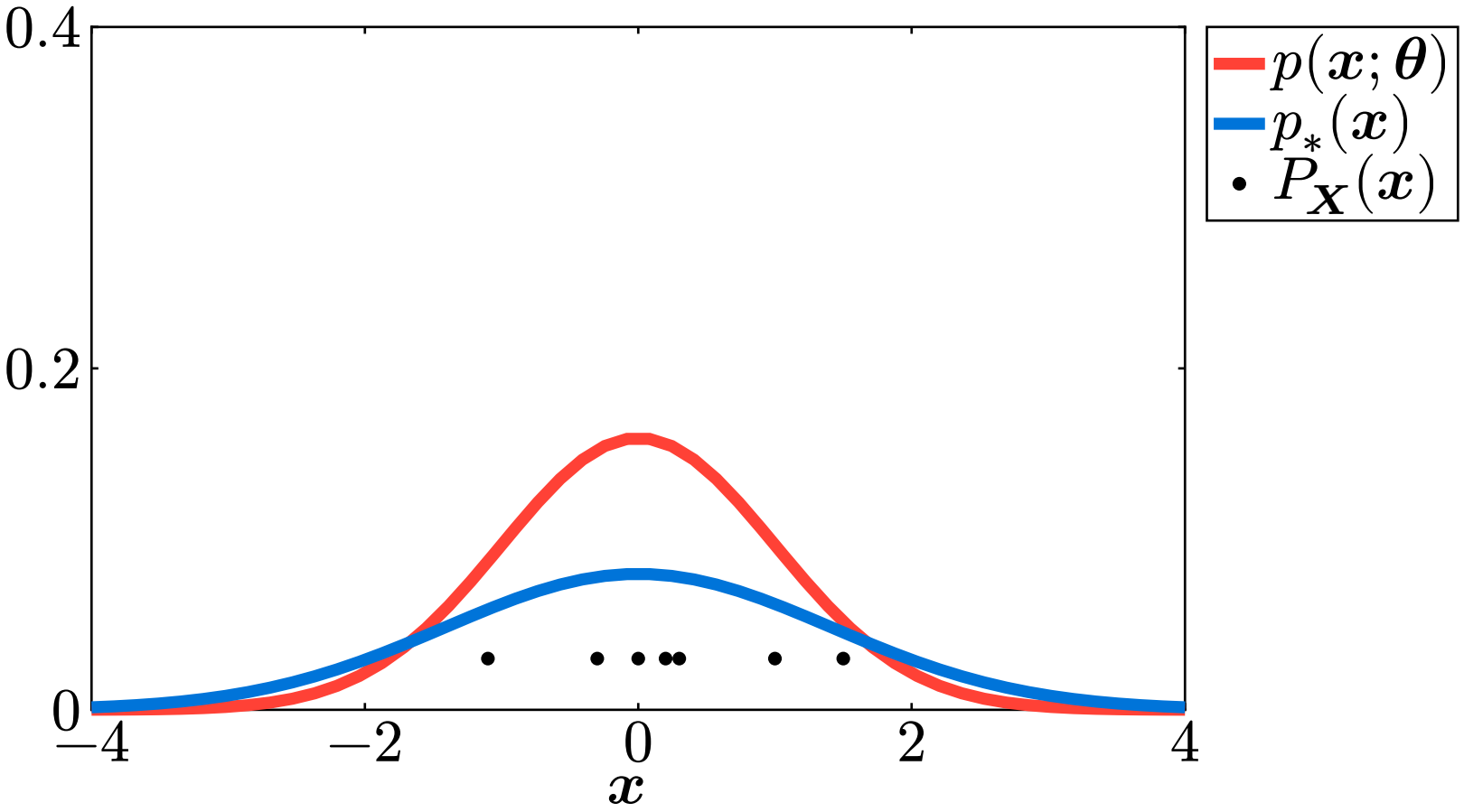
It may not seem intuitive

- More intuitive to think about equivalent maximization objective
  - $\min \Rightarrow \max, - \Rightarrow +$

$$\arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{n} \log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)$$

Find $\boldsymbol{\theta}$ so likelihood p is maximized at datapoints $\boldsymbol{x}_{[i]}$

Probabilistic Generative Models



$$p : \Theta \mapsto \Delta(X)$$

Probabilistic Generative Models

To summarize, probabilistic generative models:

1. Use a finite dataset distribution $P_{\boldsymbol{X}}(\boldsymbol{x})$
2. To approximate the true data distribution $p_*(\boldsymbol{x})$
3. By learning a **continuous** distribution $p(\boldsymbol{x};\boldsymbol{\theta})$
4. That minimizes some approximation of negative log likelihood

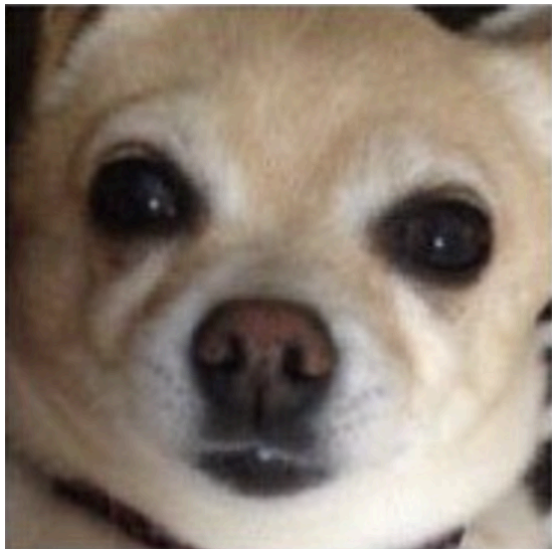
Generative model $p(\boldsymbol{x};\boldsymbol{\theta})$

Negative log likelihood objective $\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log p\left(\boldsymbol{x}_{[i]};\boldsymbol{\theta}\right)$

Bayesian Generative Models

Bayesian Generative Models

Let us start to implement generative models



$\boldsymbol{x}_{[i]}$

- Model $\approx p(\boldsymbol{x}; \boldsymbol{\theta})$
- Objective $\approx \arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log p\left(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\right)$

Question: How to represent $p(\boldsymbol{x}; \boldsymbol{\theta})$?

- $\boldsymbol{x}$ is very high dimensional
  - $\boldsymbol{x} \in [0, 1]^{3 \times 28 \times 28}$
- Complex relationships between pixels
  - Generally intractable to approximate $p_*(\boldsymbol{x})$

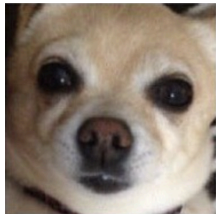
Key idea: Approximate low-dimensional representation of $\boldsymbol{x}$

- Can reason over low-dimensional representation

Bayesian Generative Models

Let $\boldsymbol{y}_{[i]}$ be a low-dimensional label of $\boldsymbol{x}_{[i]}$

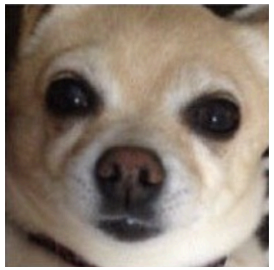
$$\boldsymbol{x}_{[i]} = \quad \boldsymbol{y}_{[i]} = \begin{bmatrix} \text{Small} \\ \text{Ugly} \end{bmatrix}$$



The label tells us which $\boldsymbol{x}$ to generate

- Many possible dog pictures, should be specific (condition on label)

$$\underbrace{\quad}_{\boldsymbol{x}_{[i]}} \sim p\left(\underbrace{\text{Small ugly dog pictures}}_{\boldsymbol{x}} \middle| \underbrace{\begin{bmatrix} \text{Small} \\ \text{Ugly} \end{bmatrix}}_{\boldsymbol{y}_{[i]}}; \boldsymbol{\theta} \right)$$



Bayesian Generative Models

We must consider all possible $\boldsymbol{y}$ to represent all possible dog pictures

$$\begin{bmatrix}\text{Small}\\ \text{Ugly}\end{bmatrix}, \begin{bmatrix}\text{Medium}\\ \text{Ugly}\end{bmatrix}, \begin{bmatrix}\text{Big}\\ \text{Not ugly}\end{bmatrix}, \cdots$$

We also need to know the label distribution $p(\boldsymbol{y})$

- How common or rare are ugly dogs? Big dogs? Small dogs?

$$p(\boldsymbol{x};\boldsymbol{\theta}) = \int \underbrace{p(\boldsymbol{x} \mid \boldsymbol{y};\boldsymbol{\theta})}_{\text{Image from label}} \overbrace{p(\boldsymbol{y})}^{\text{Label distribution}} \mathrm{d}\boldsymbol{y}$$

This is the definition of a conditional distribution

Bayesian Generative Models

Bayesian generative model:

$$p(\boldsymbol{x};\boldsymbol{\theta}) = \int p(\boldsymbol{x} \mid \boldsymbol{y};\boldsymbol{\theta}) \cdot p(\boldsymbol{y}) \, \mathrm{d}\boldsymbol{y}$$

Question: What was the objective?

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log p\Big(\boldsymbol{x}_{[i]};\boldsymbol{\theta}\Big)$$

Can plug model into objective

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log \int p\Big(\boldsymbol{x}_{[i]};\mid \boldsymbol{y};\boldsymbol{\theta}\Big) \cdot p(\boldsymbol{y}) \, \mathrm{d}\boldsymbol{y}$$

Bayesian Generative Models

$$\underbrace{\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log \overbrace{\int p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{y}; \boldsymbol{\theta}\right) \cdot p(\boldsymbol{y}) \, \mathrm{d}\boldsymbol{y}}^{\text{Model}}}_{\text{Objective}}$$

Aftering learning $p(\boldsymbol{x} \mid \boldsymbol{y}; \boldsymbol{\theta})$ we can generate new datapoints

1. Either choose or sample label $\boldsymbol{y}_{[i]} \sim p(\boldsymbol{y})$
2. Predict conditional distribution $p\left(\boldsymbol{x} \mid \boldsymbol{y}_{[i]}; \boldsymbol{\theta}\right)$
3. Sample from conditional distribution $\boldsymbol{x}_{[i]} \sim p\left(\boldsymbol{x} \mid \boldsymbol{y}_{[i]}; \boldsymbol{\theta}\right)$

Very easy! But this requires labels $\boldsymbol{y}$

- Can we do this unsupervised (without labels/humans)?

Variational Autoencoders

Variational Autoencoders

Consider a generative model **without labels $\boldsymbol{y}$**

- We will **learn** a low-dimensional latent description $\boldsymbol{z}$ instead
  - $\boldsymbol{z}$ encodes the structure of the dataset (and the world)
  - Replace label distribution $p(\boldsymbol{y})$ with latent distribution $p(\boldsymbol{z})$

$$p(\boldsymbol{x};\boldsymbol{\theta}) = \int p(\boldsymbol{x} \mid \boldsymbol{y};\boldsymbol{\theta}) \cdot p(\boldsymbol{y}) \, \mathrm{d}\boldsymbol{y}$$

$$p(\boldsymbol{x};\boldsymbol{\theta}) = \int p(\boldsymbol{x} \mid \boldsymbol{z};\boldsymbol{\theta}) \cdot p(\boldsymbol{z}) \, \mathrm{d}\boldsymbol{z}$$

We can choose this distribution, Gaussian, Bernoulli, ...

- For now, make it easy $p(\boldsymbol{z}) = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$

Variational Autoencoders

Question: How can we learn structure of $\boldsymbol{z}$?

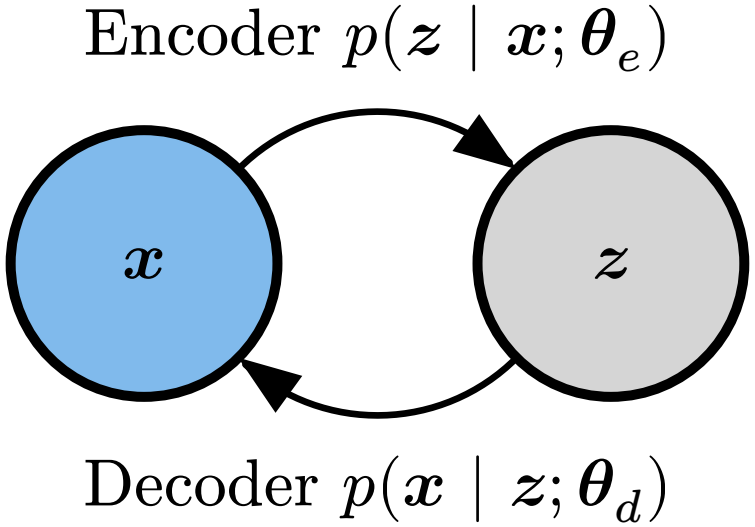
Answer: Autoencoding!

$$\hat{\boldsymbol{x}} = f^{-1}\left(\underbrace{f(\boldsymbol{x}, \boldsymbol{\theta}_e)}_{\boldsymbol{z}}, \boldsymbol{\theta}_d\right)$$

But our models are probabilistic, so it is more complex

Variational Autoencoders

$$p(\boldsymbol{x};\boldsymbol{\theta}) = \int p(\boldsymbol{x} \mid \boldsymbol{z};\boldsymbol{\theta})p(\boldsymbol{z})\,\mathrm{d}\boldsymbol{z}$$



1. Use encoder and decoder to learn latent structure $\boldsymbol{z}$
2. After learning, delete encoder (encoder only needed to learn $\boldsymbol{z}$)
3. Use decoder and latent distribution for generation

Sound too easy? It is, making it tractable is much harder

Practical Considerations

Practical Considerations

Most generative models follow a similar approach

$$\underbrace{\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log \overbrace{\int p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}\right) \cdot p(\boldsymbol{z}) \, \mathrm{d}\boldsymbol{z}}^{\text{Model}}}_{\text{Objective}}$$

However, this expression is often intractable

Question: Why? **Answer:** Cannot solve integral

- No analytical solution if $p(\boldsymbol{x} \mid \boldsymbol{z}; \boldsymbol{\theta})$ is neural network
- Indefinite integration over continuous/infinite variable $\boldsymbol{z}$

Practical Considerations

$$\underbrace{\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log \overbrace{\int p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}\right) \cdot p(\boldsymbol{z}) \, \mathrm{d}\boldsymbol{z}}^{\text{Model}}}_{\text{Objective}}$$

Variational inference methods (like VAEs) approximate this objective using the **evidence lower bound** (ELBO)

- Lower bounds the true log likelihood

The derivation is interesting, but requires concepts you do not know

- Expected values, Jensen's inequality, etc
- Instead, I will show you the final result

Practical Considerations

The original objective

$$\underbrace{\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log \overbrace{\int p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}\right) \cdot p(\boldsymbol{z}) \, \mathrm{d}\boldsymbol{z}}^{\text{Model}}}_{\text{Objective}}$$

The surrogate objective (ELBO)

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z} \sim p\left(\boldsymbol{z} \mid \boldsymbol{x}_{[i]}; \boldsymbol{\theta}_e\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}_d\right)\right] + \mathrm{KL}(p(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\theta}_e), p(\boldsymbol{z}))$$

This is a **variational autoencoder** (VAE)

Practical Considerations

Encoder

Decoder

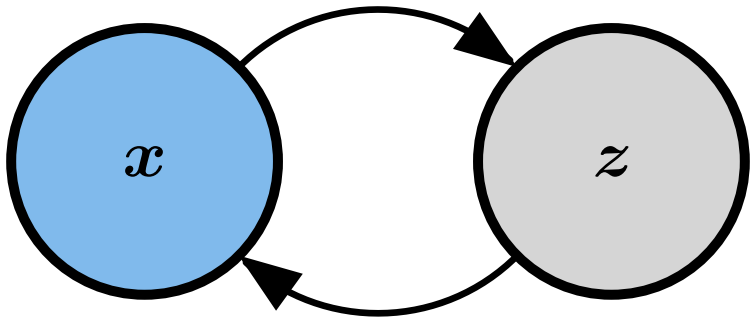
Encoder

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z}\sim p\left(\boldsymbol{z} \mid \boldsymbol{x}_{[i]};\boldsymbol{\theta}_e\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z};\boldsymbol{\theta}_d\right)\right] + \mathrm{KL}\left(p\left(\boldsymbol{z} \mid \boldsymbol{x}_{[i]};\boldsymbol{\theta}_e\right), p(\boldsymbol{z})\right)$$

Reconstruction

Marginal matching

Encoder $p(\boldsymbol{z} \mid \boldsymbol{x};\boldsymbol{\theta}_e)$



Decoder $p(\boldsymbol{x} \mid \boldsymbol{z};\boldsymbol{\theta}_d)$

Implementation

Implementation

If you did not understand the theory, it is ok

But pay attention now, we will implement the VAE

- Will use autoencoder notation to simplify when possible

Implementation

How do we implement encoder f (i.e., $p(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\theta})$)?

$$f : X \times \Theta \mapsto \Delta Z$$

Recall we let $p(\boldsymbol{z}) = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$

- Encoder should output normal distribution so KL term is analytical
- Normal distribution has a mean $\mu \in \mathbb{R}$ and standard deviation $\sigma \in \mathbb{R}_+$
- Our encoder should output d_z means and d_z standard deviations

$$f : X \times \Theta \mapsto \mathbb{R}^{d_z} \times \mathbb{R}_+^{d_z}$$

Implementation

```
core = nn.Sequential(...)
mu_layer = nn.Linear(d_h, d_z)
# Neural networks output real numbers
# But sigma must be positive
# Output log sigma, because e^(sigma) is always positive
log_sigma_layer = nn.Linear(d_h, d_z)

tmp = core(x)
mu = mu_layer(tmp)
log_sigma = log_sigma_layer(tmp)
distribution = (mu, exp(sigma))
```

Implementation

We covered the encoder

$$f : X \times \Theta \mapsto \Delta Z$$

We can use the same decoder as a standard autoencoder

$$f^{-1} : Z \times \Theta \mapsto X$$

Question: Any issues?

Answer: Encoder outputs a distribution ΔZ but decoder input is Z

Implementation

We can sample from the distribution

$$\boldsymbol{\mu}, \boldsymbol{\sigma} = f(\boldsymbol{x}, \boldsymbol{\theta}_e)$$
$$\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma})$$

But there is a problem! Sampling is not differentiable

Question: Why does this matter?

Answer: Must be differentiable for gradient descent

Implementation

VAE paper proposes the **reparameterization trick**

$$\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma})$$

$$\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{1})$$

Gradient can flow through $\boldsymbol{\mu}, \boldsymbol{\sigma}$

We can sample and use gradient descent

Implementation

Put it all together

Step 1: Encode the input to a normal distribution

$$\boldsymbol{\mu}, \boldsymbol{\sigma} = f(\boldsymbol{x}, \boldsymbol{\theta}_e)$$

Step 2: Generate a sample from distribution

$$\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$$

Step 3: Decode the sample

$$\hat{\boldsymbol{x}} = f^{-1}(\boldsymbol{z}, \boldsymbol{\theta}_d)$$

Implementation

Last thing, implement the loss function (ELBO)

Decoder	Encoder	Marginal
$p(\boldsymbol{x} \mid \boldsymbol{z}; \boldsymbol{\theta}_d)$	$p(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\theta}_e)$	$p(\boldsymbol{z}) = \mathcal{N}(\mathbf{0}, \mathbf{1})$

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z} \sim p\left(\boldsymbol{z} \mid \boldsymbol{x}_{[i]}; \boldsymbol{\theta}_e\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}_d\right) \right] + \mathrm{KL}(p(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\theta}_e), p(\boldsymbol{z}))$$

Rewrite objective ass loss function for gradient descent

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z} \sim p\left(\boldsymbol{z} \mid \boldsymbol{x}_{[i]}; \boldsymbol{\theta}_e\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}_d\right) \right] + \mathrm{KL}(p(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\theta}_e), p(\boldsymbol{z}))$$

Implementation

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z} \sim p\left(\boldsymbol{z} \mid \boldsymbol{x}_{[i]}; \boldsymbol{\theta}_e\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}_d\right) \right] + \mathrm{KL}(p(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\theta}_e), p(\boldsymbol{z}))$$

First, rewrite KL term using our encoder f

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z} \sim p\left(\boldsymbol{z} \mid \boldsymbol{x}_{[i]}; \boldsymbol{\theta}_e\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}_d\right) \right] + \mathrm{KL}(f(\boldsymbol{x}, \boldsymbol{\theta}_e), p(\boldsymbol{z}))$$

$p(\boldsymbol{z})$ and $f(\boldsymbol{x}, \boldsymbol{\theta}_e) = \boldsymbol{\mu}, \boldsymbol{\sigma}$ are Gaussian, we can simplify KL term

$$= \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z} \sim p\left(\boldsymbol{z} \mid \boldsymbol{x}_{[i]}; \boldsymbol{\theta}_e\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}_d\right) \right] + 0.5 \left(\sum_{j=1}^{d_z} \mu_j^2 + \sigma_j^2 - \log(\sigma^2) - 1 \right)$$

Implementation

$$= \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z} \sim p\left(\boldsymbol{z} \mid \boldsymbol{x}_{[i]}; \boldsymbol{\theta}_e\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}_d\right) \right] + 0.5 \left(\sum_{j=1}^{d_z} \mu_j^2 + \sigma_j^2 - \log(\sigma^2) - 1 \right)$$

Log probability for unit Gaussian is just mean square error

$$\log p \propto \log \exp\left((x - \mu)^2\right) = (x - \mu)^2$$

$$\sum_{i=1}^{n} \left(\sum_{j=1}^{d_x} \left(x_{[i],j} - f^{-1}\left(f\left(\boldsymbol{x}_{[i]}, \boldsymbol{\theta}_e \right), \boldsymbol{\theta}_d \right)_j \right)^2 - 0.5 \sum_{j=1}^{d_z} \left(\mu_j^2 + \sigma_j^2 - \log\left(\sigma_j^2\right) - 1 \right) \right)$$

Implementation

$$\sum_{i=1}^{n}\left(\sum_{j=1}^{d_x}\left(x_{[i],j}-f^{-1}\left(f\left(\boldsymbol{x}_{[i]},\boldsymbol{\theta}_e\right),\boldsymbol{\theta}_d\right)_j\right)^2-0.5\sum_{j=1}^{d_z}\left(\mu_j^2+\sigma_j^2-\log\left(\sigma_j^2\right)-1\right)\right)$$

Scale of two terms can vary, we do not want one term to dominate

- Modern VAEs introduce a hyperparameter β

$$\sum_{i=1}^{n}\left(\sum_{j=1}^{d_x}\left(x_{[i],j}-f^{-1}\left(f\left(\boldsymbol{x}_{[i]},\boldsymbol{\theta}_e\right),\boldsymbol{\theta}_d\right)_j\right)^2-0.5\beta\sum_{j=1}^{d_z}\left(\mu_j^2+\sigma_j^2-\log\left(\sigma_j^2\right)-1\right)\right)$$

Implementation

```
import torch

def L(model, x, beta):
    mu, sigma = model.f(x)
    epsilon = torch.randn_like(mu)
    z = mu + sigma * epsilon

    pred_x = model.f_inverse(z)
    recon = torch.sum((x - pred_x) ** 2)

    kl = torch.sum(
        mu ** 2 + sigma ** 2 - torch.log(sigma ** 2) - 1
    )
    return recon + beta * kl
```

Hierarchical VAEs

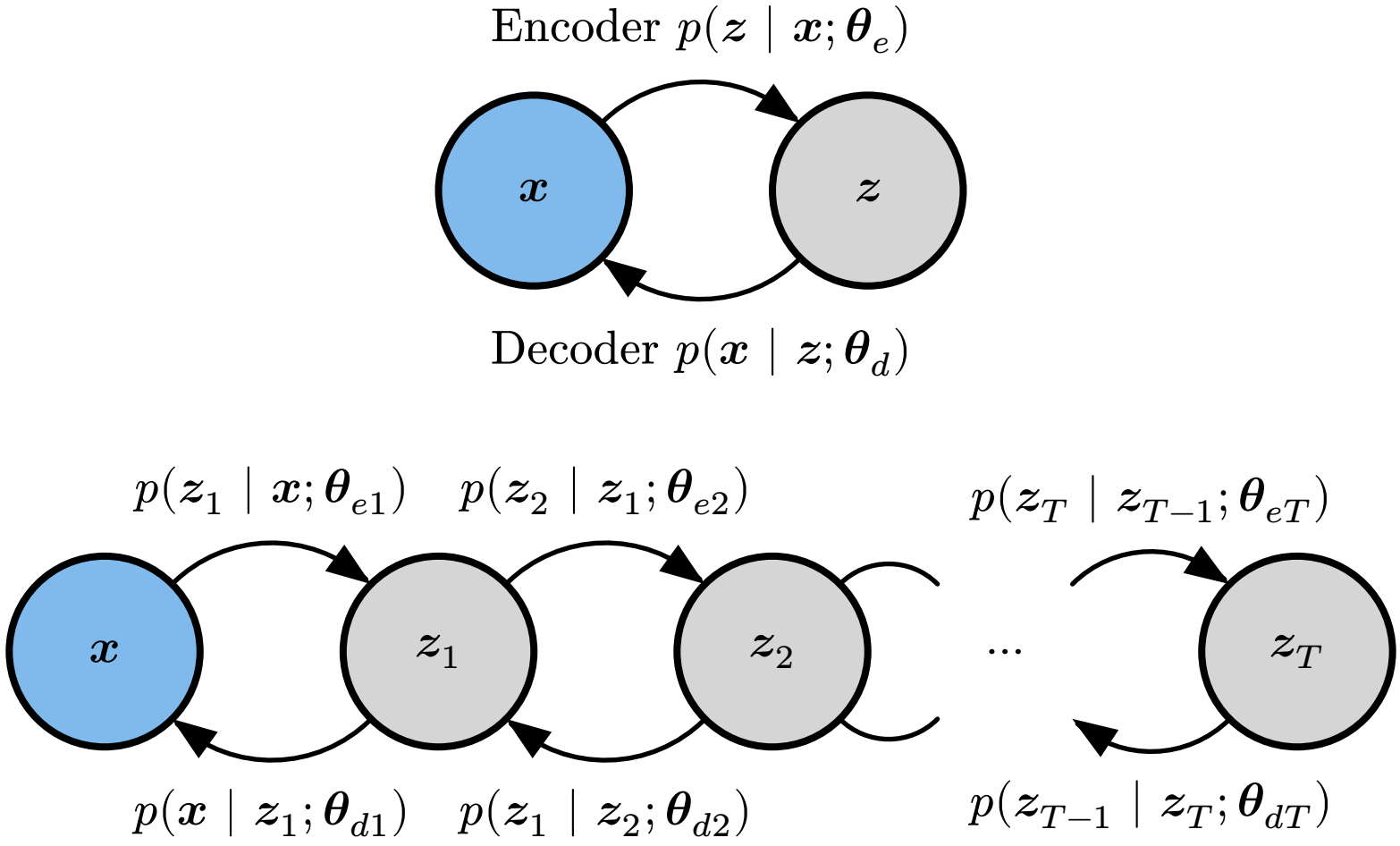
Hierarchical VAEs

VAEs are just too easy

Some crazy people decided to make them more complex

- **Hierarchical VAEs** are many VAEs stacked together
  - Provide better results than VAEs

Hierarchical VAEs



Hierarchical VAEs

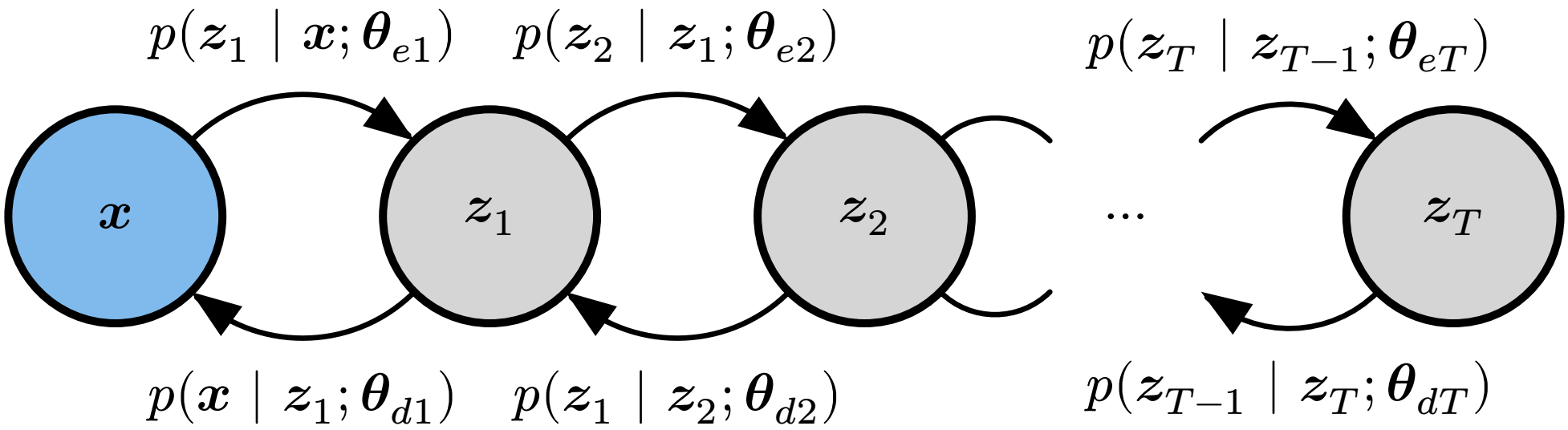
$$\mathcal{L}_{\text{VAE}}(\boldsymbol{X}, \boldsymbol{\theta}) = \sum_{i=1}^{n} \underbrace{\mathbb{E}_{\boldsymbol{z} \sim p\left(\boldsymbol{z} \mid \boldsymbol{x}_{[i]}; \boldsymbol{\theta}_e\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}; \boldsymbol{\theta}_d\right) \right]}_{\text{Reconstruction}} + \underbrace{\mathrm{KL}(p(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\theta}_e), p(\boldsymbol{z}))}_{\text{Marginal match}}$$

$$\mathcal{L}_{H} = \sum_{i=1}^{n} \underbrace{\mathbb{E}_{\boldsymbol{z}_1 \sim p\left(\boldsymbol{z}_1 \mid \boldsymbol{x}_{[i]}; \boldsymbol{\theta}_{e1}\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}_1; \boldsymbol{\theta}_{d1}\right) \right]}_{\text{Reconstruction}} + \underbrace{\mathrm{KL}(p(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\theta}_{e1}), p(\boldsymbol{z}))}_{\text{Mariginal match}}$$

$$+ \sum_{t=2}^{T} \underbrace{\mathrm{KL}\left(p(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}; \boldsymbol{\theta}_{et}), p\left(\boldsymbol{z}_t \mid \boldsymbol{z}_{t+1}; \boldsymbol{\theta}_{dt}\right)\right)}_{\text{Consistency}}$$

Hierarchical VAEs

$$\sum_{t=2}^{T} \underbrace{\mathrm{KL}\big(p(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}; \boldsymbol{\theta}_{et}), p(\boldsymbol{z}_t \mid \boldsymbol{z}_{t+1}; \boldsymbol{\theta}_{dt})\big)}_{\text{Consistency}}$$



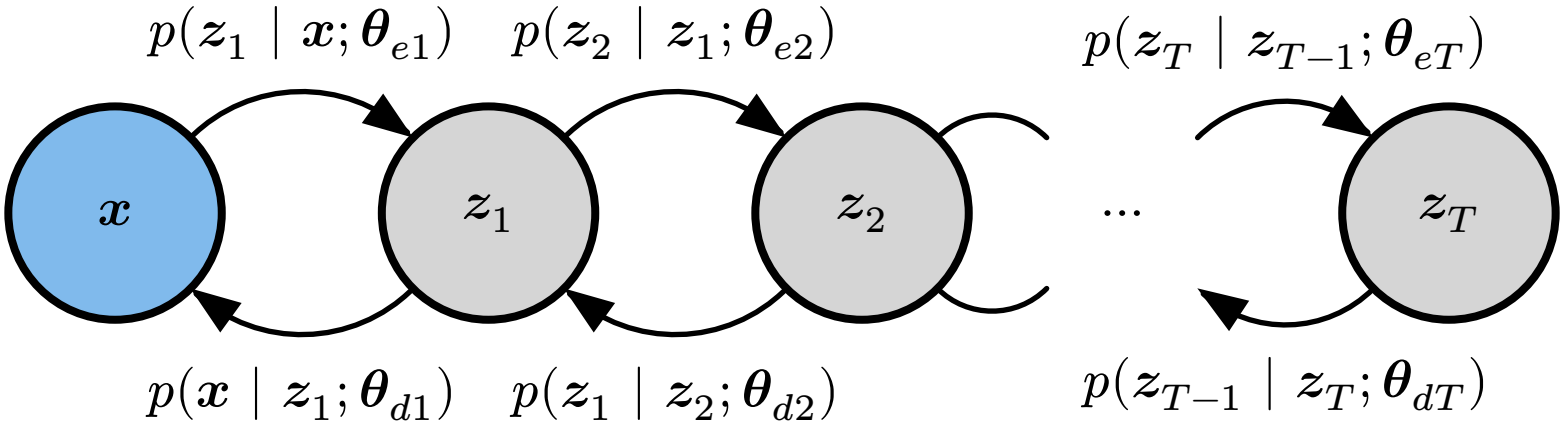
Diffusion Models

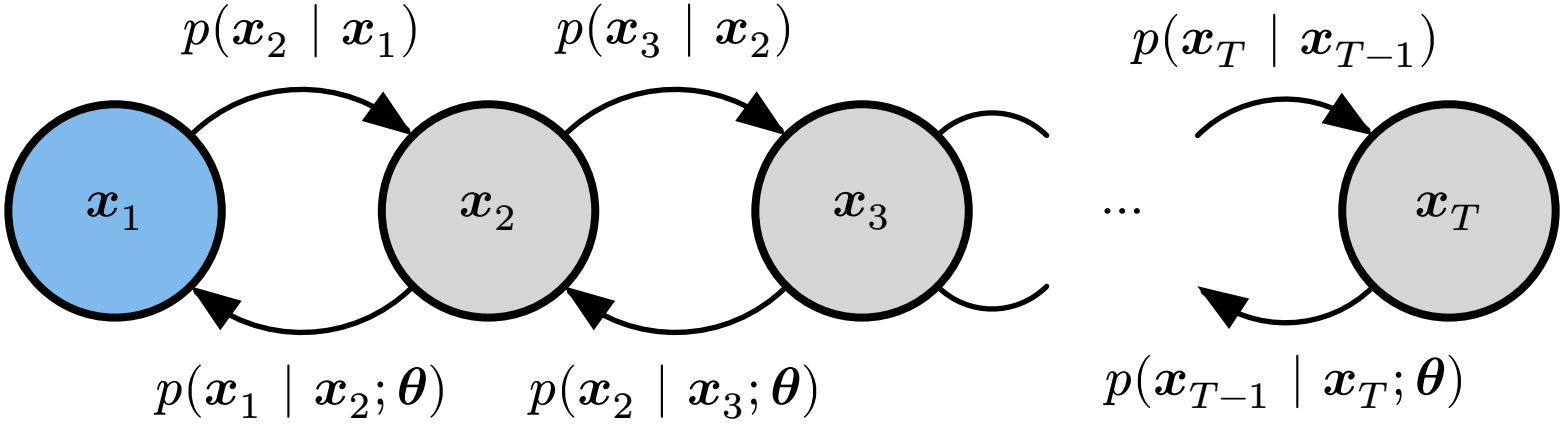
Diffusion Models

Diffusion models are simplified HVAEs

- Instead of learning encoder, use fixed encoder
  - Operates in input space X instead of latent space Z
- Share parameters for all decoders

Diffusion Models





Diffusion Models

$$\mathcal{L}_{\mathrm{H}} = \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z}_1 \sim p\left(\boldsymbol{z}_1 \mid \boldsymbol{x}_{[i]}; \boldsymbol{\theta}_{e1}\right)} \left[-\log p\left(\boldsymbol{x}_{[i]} \mid \boldsymbol{z}_1; \boldsymbol{\theta}_{d1}\right) \right] + \mathrm{KL}(p(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\theta}_{e1}), p(\boldsymbol{z}))$$

$$+ \sum_{t=2}^{T} \mathrm{KL}(p(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}; \boldsymbol{\theta}_{et}), p(\boldsymbol{z}_t \mid \boldsymbol{z}_{t+1}; \boldsymbol{\theta}_{dt}))$$

$$\mathcal{L}_{\mathrm{Dif}} = \sum_{i=1}^{n} \underbrace{\mathbb{E}_{\boldsymbol{z}_1 \sim p\left(\boldsymbol{x}_2 \mid \boldsymbol{x}_{1,[i]}\right)} \left[-\log p\left(\boldsymbol{x}_{1,[i]} \mid \boldsymbol{x}_2; \boldsymbol{\theta}\right) \right]}_{\text{Reconstruction}}$$

$$+ \sum_{t=2}^{T} \underbrace{\mathrm{KL}(p(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}), p(\boldsymbol{x}_t \mid \boldsymbol{x}_{t+1}; \boldsymbol{\theta}))}_{\text{Consistency (Denoising)}}$$

Diffusion Models

What is encoder $p(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1})$?

Gaussian noise

$$p(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \boldsymbol{x}_{t-1} + \mathcal{N}(\mathbf{0}, \boldsymbol{I})$$

It is actually slightly more complicated, but same idea

$$p(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} \cdot \boldsymbol{x}_{t-1}, (1 - \alpha_t) \cdot \boldsymbol{I})$$

Question: How is this different than VAE encoder?

- VAE **learns** to output $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$, diffusion model outputs fixed $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$
  - VAE noise in latent space, diffusion in input space
- Hierarchical VAEs strictly more powerful than diffusion models

Diffusion Models

How should I choose VAE/HVAE/diffusion model?

Choose VAE for:

- Low-dimensional representation $\boldsymbol{z}$
- Simpler problem

Choose HVAE for:

- Low-dimensional representation $\boldsymbol{z}$
- Harder problem

Choose diffusion model if:

- Do not care about $\boldsymbol{z}$
- Have 16 GPUs